

A10-C submission

- title: A10-C submission

question 1

Used sounds:

- A10-b-1.wav is re-synthesized from: Small String Orchestra Tuning by unfa -- <https://freesound.org/s/170857/> -- License: Creative Commons 0
- A10-b-2.wav is re-synthesized from: orchestra tuning+applause by FrancoD -- <https://freesound.org/s/739048/> -- License: Creative Commons 0

Both sounds are similar in the instruments featured (except for the clapping sound for the second sound), so I use the same parameters for both sounds

- [0 Hz, 500 Hz): M=5292, N=8192
- [500 Hz, 2500 Hz): M=2646, N=4096
- [2500 Hz, 22050 Hz): M=1323, N=2048

A Blackman window is used for its relatively low noise and acceptable main lobe size. The middle window size (M=2646) is chosen by noticing that 100 Hz is a good resolution for the middle band. So we calculate $6 \cdot 44100 / 100 = 2646$. The remaining two bands are derived by respectively multiplying and dividing this number by 2. The band boundaries are chosen by inspecting the spectrogram, and find that these boundaries can somewhat segment the spectrogram (though the boundaries are not very clear).

Comparing with the sounds re-synthesized using the normal sine model, I see the spectrogram using multi-resolution can capture more sinusoids at higher frequencies. This is because shorter notes at high frequency can be captured by a smaller window size at higher frequencies. The clapping sound, which is hard to re-synthesize well, is reproduced better using multi-resolution. This shows the time—frequency tradeoff is balanced better in the multi-resolution model. The drawback is that the computation complexity is much higher. This is because we need to apply DFT 3 times per frame with the multi-resolution model.

If I were to extend it to the HPR and/or HPS model, the challenge would be the method to identify fundamental frequencies and harmonics across the 3 bands. Also, the harmonics can change in between bands over time, so we also need to take this into account.

-
- file: [submission.pdf](#)
 - source: [submission.md](#)

question 2

- file: [part 2.zip](#)
 - source: [part 2/](#)
 - [170857unfasmall-string-orchestra-tuning.flac](#)
 - [739048francodorchestra-tuningapplause.wav](#)
 - [A10-b-1-sineModel.wav](#)
 - [A10-b-1-source.wav](#)
 - [A10-b-1.wav](#)
 - [A10-b-2-sineModel.wav](#)
 - [A10-b-2-source.wav](#)
 - [A10-b-2.wav](#)

question 3

```
from numpy.typing import NDArray
from scipy.fft import ifft
from scipy.signal.windows import blackman, blackmanharris, triang
from smstools.models import dftModel as DFT, utilFunctions as UF

import math
import numpy as np

def sineModel(x, fs, w, N, t):
    """
    Analysis/synthesis of a sound using the sinusoidal model, without sine
    tracking
    x: input array sound, w: analysis window, N: size of complex spectrum,
    t: threshold in negative dB
    returns y: output array sound
    """

    hM1 = int(math.floor((w.size + 1) / 2)) # half analysis window size by
rounding
    hM2 = int(math.floor(w.size / 2)) # half analysis window size by floor
    Ns = 512 # FFT size for synthesis (even)
    H = Ns // 4 # Hop size used for analysis and synthesis
    hNs = Ns // 2 # half of synthesis FFT size
    pin = max(hNs, hM1) # init sound pointer in middle of anal window
    pend = x.size - max(hNs, hM1) # last sample to start a frame
    yw = np.zeros(Ns) # initialize output sound frame
    y = np.zeros(x.size) # initialize output array
    w = w / sum(w) # normalize analysis window
    sw = np.zeros(Ns) # initialize synthesis window
    ow = triang(2 * H) # triangular window
```

```

sw[hNs - H : hNs + H] = ow # add triangular window
bh = blackmanharris(Ns) # blackmanharris window
bh = bh / sum(bh) # normalized blackmanharris window
sw[hNs - H : hNs + H] = (
    sw[hNs - H : hNs + H] / bh[hNs - H : hNs + H]
) # normalized synthesis window
while pin < pend: # while input sound pointer is within sound
    # -----analysis-----
    x1 = x[pin - hM1 : pin + hM2] # select frame
    mX, pX = DFT.dftAnal(x1, w, N) # compute dft
    ploc = UF.peakDetection(mX, t) # detect locations of peaks
    iploc, ipmag, ipphase = UF.peakInterp(
        mX, pX, ploc
    ) # refine peak values by interpolation
    ipfreq = fs * iploc / float(N) # convert peak locations to Hertz
    # -----synthesis-----
    Y = UF.genSpecSines(
        ipfreq, ipmag, ipphase, Ns, fs
    ) # generate sines in the spectrum
    fftbuffer = np.real(iff(Y)) # compute inverse FFT # type: ignore
    yw[: hNs - 1] = fftbuffer[hNs + 1 :] # undo zero-phase window
    yw[hNs - 1 :] = fftbuffer[: hNs + 1]
    y[pin - hNs : pin + hNs] += sw * yw # overlap-add and apply a
synthesis window
    pin += H # advance sound pointer
return y

def sineModelMultiRes(
    x: NDArray[np.float64],
    fs: int,
    ww: tuple[NDArray[np.float64], NDArray[np.float64],
    NDArray[np.float64]],
    NN: tuple[int, int, int],
    t: float,
    BB: tuple[float, float, float],
):
    """
    Analysis/synthesis of a sound using the sinusoidal model, without sine
    tracking
    x: input array sound, fs: sample rate, ww: 3 analysis window, NN: 3
    sizes of complex spectrum, t: threshold in negative dB, BB: upper bounds
    (exclusive) of 3 frequency bands
    returns y: output array sound
    """
    w1, w2, w3 = ww
    N1, N2, N3 = NN
    B1, B2, B3 = BB

    hM11, hM12, hM13 = (

```

```

        int(math.floor((w1.size + 1) / 2)),
        int(math.floor((w2.size + 1) / 2)),
        int(math.floor((w3.size + 1) / 2)),
    ) # half analysis window size by rounding
    hM21, hM22, hM23 = (
        int(math.floor(w1.size / 2)),
        int(math.floor(w2.size / 2)),
        int(math.floor(w3.size / 2)),
    ) # half analysis window size by floor
    Ns = 512 # FFT size for synthesis (even)
    H = Ns // 4 # Hop size used for analysis and synthesis
    hNs = Ns // 2 # half of synthesis FFT size
    pin = max(hNs, hM11, hM12, hM13) # init sound pointer in middle of anal
window
    pend = x.size - max(hNs, hM11, hM12, hM13) # last sample to start a
frame
    yw = np.zeros(Ns) # initialize output sound frame
    y = np.zeros(x.size) # initialize output array
    w1, w2, w3 = w1 / sum(w1), w2 / sum(w2), w3 / sum(w3) # normalize
analysis window
    sw = np.zeros(Ns) # initialize synthesis window
    ow = triang(2 * H) # triangular window
    sw[hNs - H : hNs + H] = ow # add triangular window
    bh = blackmanharris(Ns) # blackmanharris window
    bh = bh / sum(bh) # normalized blackmanharris window
    sw[hNs - H : hNs + H] = (
        sw[hNs - H : hNs + H] / bh[hNs - H : hNs + H]
    ) # normalized synthesis window
    while pin < pend: # while input sound pointer is within sound
        # -----analysis-----
        x1, x2, x3 = (
            x[pin - hM11 : pin + hM21],
            x[pin - hM12 : pin + hM22],
            x[pin - hM13 : pin + hM23],
        ) # select frame
        (mX1, pX1), (mX2, pX2), (mX3, pX3) = (
            DFT.dftAnal(x1, w1, N1),
            DFT.dftAnal(x2, w2, N2),
            DFT.dftAnal(x3, w3, N3),
        ) # compute dft
        ploc1, ploc2, ploc3 = (
            UF.peakDetection(mX1, t),
            UF.peakDetection(mX2, t),
            UF.peakDetection(mX3, t),
        ) # detect locations of peaks
        (
            (iploc1, ipmag1, ipphase1),
            (iploc2, ipmag2, ipphase2),
            (iploc3, ipmag3, ipphase3),
        ) = (

```

```

        UF.peakInterp(mX1, pX1, ploc1),
        UF.peakInterp(mX2, pX2, ploc2),
        UF.peakInterp(mX3, pX3, ploc3),
    ) # refine peak values by interpolation
    ipfreq1, ipfreq2, ipfreq3 = (
        fs * iploc1 / float(N1),
        fs * iploc2 / float(N2),
        fs * iploc3 / float(N3),
    ) # convert peak locations to Hertz
    (
        ipfreq1,
        ipmag1,
        ipphase1,
        ipfreq2,
        ipmag2,
        ipphase2,
        ipfreq3,
        ipmag3,
        ipphase3,
    ) = (
        ipfreq1[(0 <= ipfreq1) & (ipfreq1 < B1)],
        ipmag1[(0 <= ipfreq1) & (ipfreq1 < B1)],
        ipphase1[(0 <= ipfreq1) & (ipfreq1 < B1)],
        ipfreq2[(B1 <= ipfreq2) & (ipfreq2 < B2)],
        ipmag2[(B1 <= ipfreq2) & (ipfreq2 < B2)],
        ipphase2[(B1 <= ipfreq2) & (ipfreq2 < B2)],
        ipfreq3[(B2 <= ipfreq3) & (ipfreq3 < B3)],
        ipmag3[(B2 <= ipfreq3) & (ipfreq3 < B3)],
        ipphase3[(B2 <= ipfreq3) & (ipfreq3 < B3)],
    )
    # -----synthesis-----
    Y1, Y2, Y3 = (
        UF.genSpecSines(ipfreq1, ipmag1, ipphase1, Ns, fs),
        UF.genSpecSines(ipfreq2, ipmag2, ipphase2, Ns, fs),
        UF.genSpecSines(ipfreq3, ipmag3, ipphase3, Ns, fs),
    ) # generate sines in the spectrum
    fftbuffer = np.real(iff(Y1 + Y2 + Y3)) # compute inverse FFT #
type: ignore
    yw[: hNs - 1] = fftbuffer[hNs + 1 :] # undo zero-phase window
    yw[hNs - 1 :] = fftbuffer[: hNs + 1]
    y[pin - hNs : pin + hNs] += sw * yw # overlap-add and apply a
synthesis window
    pin += H # advance sound pointer
    return y

def main() -> None:
    (fs1, x1), (fs2, x2) = UF.wavread("part 2/A10-b-1-source.wav"),
UF.wavread(
    "part 2/A10-b-2-source.wav"

```

```

)

UF.wavwrite(
    sineModel(x1, fs1, blackman(2646, False), 4096, -60),
    fs1,
    "part 2/A10-b-1-sineModel.wav",
) # 6 * 44100 / 100 = 2646
UF.wavwrite(
    sineModel(x2, fs2, blackman(2646, False), 4096, -60),
    fs2,
    "part 2/A10-b-2-sineModel.wav",
) # 6 * 44100 / 100 = 2646

UF.wavwrite(
    sineModelMultiRes(
        x1,
        fs1,
        (blackman(5292, False), blackman(2646, False), blackman(1323,
False)),
        (8192, 4096, 2048),
        -60,
        (500, 2500, 22050),
    ),
    fs1,
    "part 2/A10-b-1.wav",
) # 6 * 44100 / 100 = 2646
UF.wavwrite(
    sineModelMultiRes(
        x2,
        fs2,
        (blackman(5292, False), blackman(2646, False), blackman(1323,
False)),
        (8192, 4096, 2048),
        -60,
        (500, 2500, 22050),
    ),
    fs2,
    "part 2/A10-b-2.wav",
) # 6 * 44100 / 100 = 2646

if __name__ == "__main__":
    main()

```

- file: [sineModelMultiRes.py](#)